Signal Desk v2 — The Async Desk

Signal Desk v2 — The Async Desk

Nexus product proposal · 2026-07-31 · Council synthesis, revised after adversarial review

Status: post-beta Pro roadmap. This proposal is explicitly gated on shipping the beta first: signing/notarization, first external cohort using the Council synchronously. (The audit's security blockers — the RCE chain, first-run wall, cost-meter wiring — were closed in nexus PR #2 this week; distribution and first-cohort work remain.) Nothing here starts before that, except two prerequisites that also serve the beta: making the claude-code CLI call asynchronous, and persisting the usage meter.

The thesis

Signal Desk v2 is Nexus's asynchronous half. The Council tab is synchronous — you ask, models answer, you watch the meter. The Desk is the same engine turned inside-out: recurring and file-triggered jobs ("Routines") that run on your own keys and terminate in exactly two artifacts — **a note you read, or a decision you must answer.**

Positioning line, written to survive adversarial reading: **"Your Mac wakes up already briefed. Nothing acts on your behalf."** Not "works overnight" — a desktop app must be honest about sleep. The design makes wake-time catch-up the *designed* behavior, not an apology (see Scheduling honesty, below).

How this proposal was made

Deep research on Eden AI (unified AI API + workflow builder + smart routing + cost monitoring) → three-lens council draft (product architect, engineering feasibility against the actual codebase, operator/UX) → adversarial review → this revision. The adversary's fatal and serious critiques were absorbed, not argued with; the largest changes: re-sequenced behind beta, tagline and scheduling redesigned around sleep, scheduled routines moved to Pro, defensibility rewritten against the real competitors, security claims narrowed to what's provable, and the v1 mechanics simplified (a fork

ends a run; no mid-run resume).

What we took from Eden AI — and refused

Borrowed, adapted local-first: fallback chains (failure = routing decision) · cost dry-run before scheduling · per-routine spend attribution and guardrails · provider-error observability computed from your own run ledger.

Refused: the visual node-graph canvas (two-engineer-year product, n8n's home turf; every real solo-operator job fits one sentence — the file format is the product, a canvas is optional skin later) · publishing pipelines as hosted endpoints (Nexus is a console, not a server) · connector zoo · embedded code nodes (unattended arbitrary code is a security boundary we refuse to cross) · managed RAG (belongs to the Library layer, not this tab).

The product

1 · Routines — one sentence, one file.

A Routine is a single markdown file in the vault (`routines/morning-brief.routine.md`): YAML frontmatter + a prompt body. Frontmatter fields: `trigger` (daily HH:MM · watch:folder-glob · manual), `sources` (files, folders, URLs), `run` (chat | council), `council` (ordered fallback chain, e.g. deepseek > gemini > ollama:llama3), `brain`, `deliver` (brief | fork | both), `notify` (push | digest | silent; default digest), `budget` (per-run + per-month caps). The authoring UX is a sentence-shaped form — "When … take … run … deliver … cap $…" — five dropdowns and a prompt box; it serializes to the file, and power users hand-edit or script it. **v1 rule: one prompt, one output.** No step pipelines, no branching, no mid-run state. If someone needs a two-step chain: routine B watches routine A's output folder — with cycle protection (per-routine fire-rate limits, watch-debounce, and a loop detector) built in *before* that pattern is documented, because two vault-writing watchers pointed at each other is a spend amplifier.

2 · Scheduling honesty — designed for sleep, not around it.

A desktop app fires when the machine is awake. So: the scheduler is a wake/launch catch-up engine (run the most recent missed slot exactly once — never replay a backlog; a week-idle laptop must not burn seven budgets on Monday). The UI shows "last ran / next due / missed" truthfully. The pitch is wake-time freshness: open the lid,

the brief is already building, push arrives before your coffee. True overnight runs are an explicit opt-in ("Allow Nexus to wake this Mac" via scheduled wake), and watch-triggers + manual runs — which don't pretend to be cron — carry the primary demo weight. We do not claim server-side reliability we don't have; that axis belongs to ChatGPT Tasks, and we compete on a different one (below).

3 · The Decision Queue — the signature primitive, now with provenance.

The fork-packet mechanic (urgency, deadline, options, default-if-unanswered, resolution appended as the record) is promoted to the desk's identity. Any Routine can emit a fork when its Brain flags a human-only decision; the existing urgent-push/daily-brief/resolve machinery carries over. Revisions from adversarial review: every packet carries a **provenance field** (emitted by which routine, from which sources, or "external writer"), source-derived text renders visually quarantined (an injected webpage must not be able to dress itself as a trusted urgent decision), URLs and imperative instructions are stripped from option labels, and auto-recorded defaults are stamped `resolution: defaulted` — distinct from `resolution: human` — so downstream consumers can tell them apart. **A fork ends its run.** Resolution can trigger a fresh run via the watch mechanism; there is no checkpoint/resume machinery in v1.

4 · Fallback chains with an honest local floor.

Ordered chains with the availability-probe cache the app already maintains; failures log and advance. `localFloor` (only offered when Ollama is actually detected) appends the local model as the last link — but a floor-served result is a **clearly degraded artifact**: labeled "cloud unavailable — low-confidence local summary + raw source digest," never dressed as the real brief. A wrong brief confidently delivered at 7:45 is worse than an honest failure card; availability must not be laundered into false quality.

5 · The ledger and real budget enforcement — not theater.

Every run appends to a persisted NDJSON ledger (models, tokens, cost, latency, errors — with date rollover; the current in-memory meter is a stated prerequisite fix). Enforcement is layered where money actually moves: hard `max_tokens` ceilings passed to providers per step · a mid-run abort when the running ledger crosses the per-run cap · monthly caps that pause the routine and post a desk card (never a silent overrun, never a silent skip) · a global "Pause all background work" kill switch, always visible, also a Telegram command. Cost previews are shown as **ranges** ("$0.02–$0.15/run"), corrected by actuals after the first run — single-figure estimates on

variable-size sources are false precision. Provider-health readouts ship, but are labeled as maturing until weeks of personal data exist; they are not a launch selling point.

6 · The trust boundary, stated exactly.

Scheduled runs get zero tool access, enforced in code — they read declared sources and write only to the vault and notifications; anything that would act on the world terminates in a fork packet. We claim precisely: **"nothing acts on your behalf"** — not "impossible by construction," because (a) a fetched webpage can still try to *persuade the human* through a brief, which provenance-marking and quarantine mitigate but no architecture eliminates; and (b) if a future phase ever adds allowlisted tool steps, that stronger claim would have been a lie. The prerequisite security work (Host-header allowlist, fail-closed sandbox, approval-gated tools — landed in PR #2) is what makes running a scheduler inside this process defensible at all.

7 · First-run — the desk is alive in one sitting.

Day one shows one **sample fork packet** — a real, resolvable decision ("Which morning-brief style? [three rendered samples]") whose resolution creates the user's first routine; resolving a fork *is* the onboarding. Next to it: three starter routines, pre-configured but paused, each with a live preview from the user's actual data and a "Run once — est. $0.02–0.08" button. Delivery default is **in-app + system notifications**; Telegram is presented as an upgrade ("get it on your phone") with its multi-step setup honestly out of the 10-minute path. "Your Nexus week" (a self-brief from the app's own session/spend data) appears only once a week of data exists — an empty dashboard wearing a brief's clothes would be the exact idles-empty failure this redesign kills.

Notification contract

Push = blocking or perishable only (urgent forks, cap hits, watcher fires, the one daily brief) with a designed ceiling; badge = decisions owed, content never badges; digest = everything else, riding the existing Signal Brief with its PDF push. Defaults to digest; silence is default, escalation is opt-in. The desk is a queue you drain — target state empty, and it says so.

Free vs Pro — revised after adversarial review

The adversary's sharpest commercial point: the draft gave the flagship away and sold ceilings. Revised:

- **Free:** the Decision Queue (open packet interface, resolve-in-place, expiry) · every starter routine runnable **manually** ("Run now") with dry-run and ledger · in-app delivery.
- **Pro: scheduling itself** (daily/wake catch-up, watch triggers, URL polling) · council chains + fallback in routines · budgets/caps/auto-pause · Telegram + multi-channel delivery · full ledger history + provider health · template gallery.

The free experience is one tap: "Run now → here's your brief, it cost $0.03 of your own money, here's the receipt." The Pro pitch is one sentence: **"Now imagine never tapping it."** Automation-of-the-loop is the product; the loop itself is the demo. (The ledger file lives in the user's vault either way — we don't gate retention of a file the user owns; that would contradict our own no-lock-in position.)

Defensibility — against the real competitors

- **ChatGPT Tasks / Pulse, Perplexity Tasks** (the real threat on this axis): server-side, zero-setup, fire while your laptop sleeps. They win on reliability of firing; they cannot see your local files, cannot run your local models, meter nothing honestly (flat sub, opaque), offer one vendor's model, and produce artifacts locked in their apps. Nexus's counter-position: *your* files as sources, *your* keys with a receipt, a council with fallback instead of one model, markdown artifacts you own, and a human-decision queue neither has.
- **n8n / Zapier + AI:** real triggers and write-actions — for people who want to build automations. Nexus's desk is for people who want *briefs and decisions*, not flow-charts; our refusal of the canvas and of write-actions is the differentiation, not a gap.
- **Fluent or the next desktop council app adding a scheduler:** the mechanics are copyable in weeks; the compounding moats are the vault of owned artifacts (switching cost that belongs to the user), the honest-meter brand with no margin on tokens (a flat-sub business can copy the meter; a markup business can't), and the Decision Queue as an open format others can adopt but we define.
- **Eden AI** was the inspiration, not the competitor: it's B2B API middleware. What we kept from that comparison: local models in the chain and data-never-leaves are

structural advantages a cloud metering business cannot copy.

Build plan — revised

Gate: beta ships first. First external cohort on the synchronous Council; distribution (signing/notarization) done.

Prerequisites (also serve the beta, can start now): async claude-code CLI call (today's sync call can freeze every endpoint for minutes — unacceptable even before scheduling) · persisted usage meter with date rollover · single-flight run queue.

Phase A — one honest routine (7–10 dev-days): routines module reusing the proven signal.mjs tick+catch-up pattern (cross-platform state dir — no macOS-only paths; Windows parity is a listed column, not an afterthought) · manual + daily triggers · single model + labeled local floor · `max_tokens` ceilings + mid-run abort + persisted per-routine ledger · missed-run honesty in the UI · sample-fork onboarding. **Exit criterion: beta users run one routine daily for two weeks and trust the receipts.**

Phase B — council + queue depth (8–12 dev-days): extract the inline council loop into `runCouncil(opts, emit)` — the one large refactor; done first behind the existing endpoint with SSE-stream regression diffs — then council steps in routines · fork-emitting routines with provenance + quarantine rendering · watch triggers with cycle protection · Telegram upgrade path.

Phase B2 — deferred until pulled: cross-validation mode (defined then, not hand-waved now) · provider-health strip promotion · wake-the-Mac opt-in · template gallery.

Phase C — only with demonstrated demand: form-editor upgrades · read-only chain visualization · allowlisted, audited tool steps (which will require re-stating the trust boundary honestly) · at most 1–2 connectors named by real users.

Calendar honesty for a tiny team: Phases A+B ≈ 15–22 dev-days ≈ 5–8 calendar weeks alongside beta support. That is the cost of the differentiated core; everything the estimate-tripling features buy (canvas, connectors, code nodes) stays refused.

Top risks

1. **Sequencing pressure** — the temptation to build this before the beta cohort exists. The gate is the mitigation; this document says so on line three.

2. **Sleep-reliability perception** — mitigated by design (catch-up as the designed behavior, honest missed-run UI, wake opt-in later), not by copy.

3. **Council-loop extraction regression** — the SSE-coupled refactor; regression-diff before anything stacks on it.

4. **Runaway background spend** — layered enforcement (token ceilings, mid-run abort, cycle protection, kill switch); the one place to over-engineer.

5. **Injection-shaped content reaching the human** — provenance, quarantine rendering, stripped option labels; and the humility to not claim "impossible."

The one-paragraph pitch

Every AI app now has a chat box; the next fight is who owns the *recurring* loop. Server-side tasks (ChatGPT, Perplexity) will win the fire-at-3am axis. Nexus should not fight there. It should own the axis no one else can stand on: recurring AI work over **your local files**, through **a council of models with fallback** — local models included — on **your keys with a real receipt**, producing **artifacts you own**, and escalating anything that matters to a **human decision queue with a paper trail**. The Desk makes Nexus the first AI console that's honestly asynchronous on a desktop: it doesn't pretend to be a server — it makes your Mac wake up already briefed, and it never acts on your behalf.